Lesson 5.1 — What Humans Still Own at the Architecture Layer

Lesson 5.1 — What Humans Still Own at the Architecture Layer

> **Six categories of decision define a senior engineer's day. AI participates in all of them. AI does not own any of them.** This lesson is the map.

By the end of this lesson:

- You will know the six categories of architecture decision an interviewer is silently checking.
- For each, you will have an example, a senior soundbite, and a "where the AI helps" boundary.
- You will have a mental model for narrating architecture decisions in a system-design interview while the AI is in the room.

---

1. Why this lesson matters more than its length suggests

Most system-design interviews look the same in 2026 as they did in 2020. The whiteboard is still there. The "design X" prompt is still there. What is *new* is the unspoken question:

> *"With AI doing so much, how do you spend your time? What do you actually decide?"*

If you cannot answer that question structurally, you sound like every junior who claims to be senior. The six categories below are the answer.

---

2. The six categories

2.1. Domain modeling

The shape of the data, the entities, the relationships, and the invariants. The AI can propose a schema. **Choosing the schema** — knowing what to denormalize, what to make immutable, what to version — is yours.

2.2. Boundary placement

Where do components, services, and modules end and begin? The AI can split a file at a sensible spot. **Drawing the system's seams** — knowing which transformation belongs at the API, which at the service, which at the DB — is yours.

2.3. Failure-mode design

What happens when this fails? The AI can add a try/catch. **Designing for graceful degradation** — knowing what should retry, what should fail closed, what should fail open, what should circuit-break — is yours.

2.4. Trade-off ownership

When two designs are equally valid, which ships? The AI can list trade-offs. **Picking the trade-off** — accepting that the choice will be questioned in 6 months when reality bites — is yours.

2.5. Scale and cost

Will this hold at 100x the current load? At 0.1x? **Sizing the system, choosing the cost-vs-latency curve, knowing when premature optimization actually pays** — is yours.

2.6. Migration and rollout

How do you ship a change without breaking the world? The AI can produce a migration script. **Designing the rollout — feature flags, dual-writes, dark launches, canary fleets** — is yours.

Six categories. **Memorize the names.**

---

3. Category 1 — Domain modeling

What the AI does

> *"Define a Postgres schema for users, orders, and order line items."*

The AI gives you reasonable normalized tables. Often correct on the first try.

What only you do

- Decide whether `orders.user_id` references a *current* user or a historical snapshot. (This decision affects retention rules, GDPR deletion behavior, and analytics joins for the next 10 years.)
- Decide whether `order_line_items.price` is the *current* product price or the *price at order time*. The AI will not ask. **You ask.** The wrong answer here is a year of customer-support fires.
- Decide whether to add a `version` column on mutable rows. The AI rarely volunteers this. You volunteer it once you've been in a race-condition postmortem.
- Decide what is enforced as a database constraint, what at the application layer, and what only by convention. Each tier has consequences for development speed and data integrity.

Senior soundbite

> *"The AI gives me schemas that work for the demo. The schemas that work for five years require knowing which fields are point-in-time snapshots, which are current values, and which are derived. Those are domain decisions, not database decisions, and they have to come out of conversations with the people who own the business rules."*

Where the AI helps

- Generating the SQL once you've made the decisions.
- Listing trade-offs of normalize-vs-denormalize.
- Spotting missing indexes once you describe the access patterns.

---

4. Category 2 — Boundary placement

What the AI does

> *"Split this file into four components."*

It picks reasonable boundaries based on syntax (imports, exports, what calls what).

What only you do

- Decide which boundaries become **module boundaries** (one team owns them) vs **file boundaries** (anyone can edit). The AI cannot see the team chart.
- Decide which boundaries become **service boundaries** (separate deployable). Service boundaries cost a lot — observability, deployment, on-call. The AI does not feel the cost.
- Decide which API contracts are *stable forever* (external partners) vs *internal-only* (your own services). The AI does not know which partners you have.
- Decide where **caching** should live. The AI can add a cache anywhere. *Choosing* the layer requires understanding read/write ratios, invalidation cost, and customer-visible staleness tolerance.

Senior soundbite

> *"Boundaries are the most expensive thing in any architecture — once a service boundary exists, it is almost impossible to undo. The AI moves files around for me. Choosing where the boundaries are, knowing which seams must never be crossed and which can flex — those decisions live with us for years and are probably 30 percent of what 'architecting' actually means."*

Where the AI helps

- Generating the boilerplate when a boundary is decided (interface, adapter, contract).
- Spotting violations of an existing boundary in a PR.
- Drafting a deprecation note when a boundary changes.

---

5. Category 3 — Failure-mode design

What the AI does

> *"Add error handling to this function."*

It wraps the body in try/catch. Sometimes well, often as decoration.

What only you do

- Categorize each call as **must-succeed** (charge a credit card), **best-effort** (write to analytics), **idempotent** (retry the webhook), or **at-most-once** (send the email). The AI cannot tell.
- Decide which failures are **paged** (page the on-call), **alerted** (Slack), **logged** (Datadog), or **silent** (acceptable drop). Each tier costs human attention.
- Decide which paths have **circuit breakers**, **timeouts**, **bulkheads**, and **back-pressure**. The AI knows the names; the placement is yours.
- Decide what the user sees when something fails. A blank page? A retry button? A graceful empty state with a tooltip? The AI does not know your brand voice.

Senior soundbite

> *"Most architectures' failure modes are designed by accident — whatever the original code happened to do. Designing failure modes deliberately means knowing which calls must succeed, which can drop, which can retry. The AI helps me write the retry loop once I've decided this should be a retry loop. The decision is upstream of the code."*

Where the AI helps

- Implementing retry logic once you've specified the retry policy.
- Drafting alert rules from a runbook.
- Listing the failure modes in a checklist that you then categorize.

---

6. Category 4 — Trade-off ownership

What the AI does

> *"Compare three caching strategies."*

It produces a clean table of trade-offs. Genuinely useful.

What only you do

- Pick one. **Accountably.**
- Document why you picked it, in a way that survives the next on-call rotation when someone wonders why we're not doing the obvious other thing.
- Carry the explanation through stakeholder pushback ("can't we just…?") with confidence.
- Revisit when conditions change. The AI does not know that the cost of option A doubled this quarter.

Senior soundbite

> *"AI is a fantastic trade-off-listing machine. It is, by definition, not a trade-off-making machine. The trade-off only becomes a trade-off when someone owns the consequences. The AI doesn't carry a pager. I do."*

Where the AI helps

- Surfacing trade-offs you might miss.
- Estimating the cost / latency / complexity of each option.
- Drafting the ADR (architecture decision record) once you've chosen.

---

7. Category 5 — Scale and cost

What the AI does

> *"How would this design scale to 1M users?"*

It produces a plausible answer. Usually involving cache layers, read replicas, sharding, queues.

What only you do

- Distinguish **expected scale** (1M users) from **bursty scale** (1M concurrent in 5 minutes during a marketing event) from **degenerate scale** (one bad actor). Each one demands a different design.
- Decide which scaling strategies are *worth it* given your *current* size. Premature scale work is one of the most common forms of waste.
- Trade off cost against latency at each layer. The AI doesn't know your CFO's tolerance.
- Decide what to **shed** when overloaded. The AI does not own the customer's emotional response to a 503.

Senior soundbite

> *"The right scale design is rarely the maximum scale design. Most systems do not need horizontal sharding; most systems would benefit from a cache layer they don't have; most systems will fall over in ways the load test missed. Choosing the right level of scale investment for a given moment in a company's life — that's a judgment about* people *and* budget*, not just throughput. AI doesn't see those."*

Where the AI helps

- Sizing calculations once you've stated the load.
- Comparing AWS vs GCP vs self-hosted costs at a stated load.
- Sketching the failure modes of each scaling strategy.

---

8. Category 6 — Migration and rollout

What the AI does

> *"Generate a migration script that adds this column."*

It produces a script. Usually safe, sometimes not.

What only you do

- Decide whether to **dual-write** to old and new for a period.

- Decide whether to **backfill** the new column synchronously, asynchronously, or never.
- Decide whether to **feature-flag** the new path. The AI doesn't know your flag system.
- Decide whether to **canary** to 1%, 10%, 100%. The AI doesn't know your incident history.
- Decide what **rollback** looks like. The AI's "if it fails just rollback" is naive — some changes are unrollable once any traffic has used them.

Senior soundbite

> *"Migrations are where engineering meets reality. Every elegant migration on paper has a rollback story that gets told once and never written down. Designing the rollback before the rollout is the discipline. AI helps me with the script. The decision tree above the script is mine — and it's where most production incidents come from."*

Where the AI helps

- Generating the migration SQL or code.
- Drafting the dual-write logic once you've chosen it.
- Reviewing your runbook for missing steps.

---

9. The interview deployment

When you walk into a system-design interview, *especially* one that allows AI use, do this in the first two minutes:

1. **Read the problem twice.** Out loud.
2. **Ask 3-5 clarifying questions** that map to the six categories above. Examples:
   - *"What's the scale we're sizing for? Average and peak?"* (Category 5)
   - *"What's the failure tolerance — can the user retry, or is this a one-shot transaction?"* (Category 3)
   - *"Are there existing services this needs to integrate with, or is this greenfield?"* (Category 2)
   - *"Who maintains this — one team, or a shared service?"* (Category 2)
3. **Sketch the rough boxes** and label which decisions go in each box.

The interviewer immediately sees: this person thinks in categories. **That alone is half the battle.**

---

10. The "I'd ask the AI" moment

A common interviewer move:

> *"You can use an LLM during this interview. When would you?"*

Have a structured answer:

> *"Three places. One, listing trade-offs of options I haven't fully thought through — pure exploration. Two, drafting boilerplate once a decision is made — the schema SQL after we've chosen the schema, for instance. Three, sanity-checking my back-of-envelope math — I'd never trust the AI's numbers cold, but I'd use it to spot-check whether my throughput estimate is in the right order of magnitude. I would* not *use it for the architectural choices themselves. Those are why I'm in this interview."*

That answer:

- Names *three* concrete uses.
- Distinguishes *AI-assisted* from *AI-driven*.
- Closes with a strong line about why **you** are the one in the room.

Drill it.

---

11. The trap question — "wouldn't an AI just give me this design?"

Sometimes the interviewer plays devil's advocate:

> *"If I asked GPT-5 to design Twitter for me, wouldn't I get a reasonable answer?"*

The right response:

> *"You'd get a reasonable-sounding answer, yes. Whether it's* the right *answer for your specific situation — your traffic patterns, your budget, your team's familiarity with the tech, your existing stack, your customer mix — that part is the human work. The AI's design will look good on a whiteboard. Six months later, you*

discover the design assumed a write-heavy workload and yours is read-heavy by 100:1. Or it assumed cloud-native, and you're on-prem. The bridge from a plausible *design to* the right *design is contextual, and the context is what only humans on this team have."*

That answer **engages with the premise instead of dismissing it**. It also previews the trade-off conversations of Lesson 5.3.

---

12. The senior soundbite for this whole lesson

If you only memorize one paragraph from this lesson:

> *"At the architecture layer there are six categories of decision: domain modeling, boundary placement, failure-mode design, trade-off ownership, scale and cost, migration and rollout. AI helps me in all six — listing options, generating boilerplate, spot-checking math. AI doesn't* own *any of the six, because owning means accepting consequences, and consequences live with humans. My job at the senior level is to make those six decisions well, document them, and live with them. That's not work AI does."*

That is the answer to *"What does a senior engineer actually do all day in 2026?"* and it lands every time.

---

13. CHECK YOURSELF

- ☐ Can you list the six categories of architecture decision from memory?
- ☐ For each, can you give one concrete example and name the boundary between AI's contribution and yours?
- ☐ Can you deliver the section-10 "when would you ask the AI?" answer in under 60 seconds?
- ☐ Can you handle the section-11 trap question without dismissing or over-conceding?
- ☐ Can you deliver the section-12 senior soundbite without notes?

---

14. Where you are now

You have a structural model of architecture work that survives any specific design problem. Move on to **02-design-with-ai-walkthrough.md** — a fully worked 60-minute system-design walkthrough that puts these categories into practice.